

IT24103443

Exercise 07

1.

Program	Collectives Used	Memory Allocation	Manual Sum on Root?	Result Location
Ex 1: Bcast	MPI_Bcast	Full array on ALL	Yes (Send/Recv loop)	Root only
Ex 2: Scatter	MPI_Scatter	Full on root; chunk on all	Yes (Send/Recv loop)	Root only
Ex 3: Gather	MPI_Scatter, MPI_Gather	Full on root; chunk on all	Yes (Array loop)	Root only
Ex 4: Reduce	MPI_Scatter, MPI_Reduce	Full on root; chunk on all	No	Root only
Ex 5: Allreduce	MPI_Scatter, MPI_Allreduce	Full on root; chunk on all	No	All processes
Ex 6: Scan	MPI_Scatter, MPI_Scan	Full on root; chunk on all	No	Different per rank

2.

Exercise 01:

mpirun -n 2 ./program

Root filled array with values 1 to 1000000

Rank 0: summed indices [0, 500000) => local_sum = 125000250000

Rank 1: summed indices [500000, 1000000) => local_sum = 375000250000

[Bcast] Total sum = 500000500000

[Bcast] Expected = 500000500000

[Bcast] Correct? = YES

[Bcast] Time = 0.0016 sec

mpirun -n 4 ./program

Root filled array with values 1 to 1000000

Rank 0: summed indices [0, 250000) => local_sum = 31250125000

Rank 1: summed indices [250000, 500000) => local_sum = 93750125000

Rank 2: summed indices [500000, 750000) => local_sum = 156250125000

Rank 3: summed indices [750000, 1000000) => local_sum = 218750125000

[Bcast] Total sum = 500000500000
[Bcast] Expected = 500000500000
[Bcast] Correct? = YES
[Bcast] Time = 0.0046 sec

mpirun -n 8 ./program

Root filled array with values 1 to 1000000

Rank 7: summed indices [875000, 1000000) => local_sum = 117187562500
Rank 5: summed indices [625000, 750000) => local_sum = 85937562500
Rank 2: summed indices [250000, 375000) => local_sum = 39062562500
Rank 6: summed indices [750000, 875000) => local_sum = 101562562500
Rank 4: summed indices [500000, 625000) => local_sum = 70312562500
Rank 1: summed indices [125000, 250000) => local_sum = 23437562500
Rank 0: summed indices [0, 125000) => local_sum = 7812562500
Rank 3: summed indices [375000, 500000) => local_sum = 54687562500

[Bcast] Total sum = 500000500000
[Bcast] Expected = 500000500000
[Bcast] Correct? = YES
[Bcast] Time = 0.0075 sec

Exercise 02:

mpirun -np 2 ./sum_scatter

Root filled array with values 1 to 1000000

Rank 0: local_sum = 125000250000
Rank 1: local_sum = 375000250000

[Scatter] Total sum = 500000500000
[Scatter] Correct? = YES
[Scatter] Time = 0.0012 sec

mpirun -np 4 ./sum_scatter

Root filled array with values 1 to 1000000

Rank 2: local_sum = 156250125000
Rank 1: local_sum = 93750125000
Rank 3: local_sum = 218750125000
Rank 0: local_sum = 31250125000

[Scatter] Total sum = 500000500000
[Scatter] Correct? = YES
[Scatter] Time = 0.0058 sec

mpirun -np 8 ./sum_scatter

Root filled array with values 1 to 1000000

Rank 1: local_sum = 23437562500
Rank 6: local_sum = 101562562500
Rank 5: local_sum = 85937562500
Rank 3: local_sum = 54687562500
Rank 7: local_sum = 117187562500
Rank 2: local_sum = 39062562500
Rank 0: local_sum = 7812562500
Rank 4: local_sum = 70312562500

[Scatter] Total sum = 500000500000
[Scatter] Correct? = YES
[Scatter] Time = 0.0039 sec

Exercise 03:

mpirun -np 2 ./sum_gather

Root filled array with values 1 to 1000000
Rank 1: local sum = 375000250000
Rank 0: local sum = 125000250000

[Gather] Total sum = 500000500000
[Gather] Correct? = YES
[Gather] Time = 0.001814 seconds

mpirun -np 4 ./sum_gather

Root filled array with values 1 to 1000000

Rank 1: local sum = 93750125000

Rank 0: local sum = 31250125000

Rank 3: local sum = 218750125000

Rank 2: local sum = 156250125000

[Gather] Total sum = 500000500000

[Gather] Correct? = YES

[Gather] Time = 0.005815 seconds

mpirun -np 8 ./sum_gather

Root filled array with values 1 to 1000000

Rank 1: local sum = 23437562500

Rank 4: local sum = 70312562500

Rank 3: local sum = 54687562500

Rank 2: local sum = 39062562500

Rank 7: local sum = 117187562500

Rank 6: local sum = 101562562500

Rank 0: local sum = 7812562500

Rank 5: local sum = 85937562500

[Gather] Total sum = 500000500000

[Gather] Correct? = YES

[Gather] Time = 0.005598 seconds

Exercise 04:

mpirun -np 2 ./sum_reduce

Root filled array with values 1 to 1000000

Rank 1: local sum = 375000250000

Rank 0: local sum = 125000250000

[Reduce] Total sum = 500000500000

[Reduce] Correct? = YES

[Reduce] Time = 0.002309 seconds

mpirun -np 4 ./sum_reduce

Root filled array with values 1 to 1000000

Rank 1: local sum = 93750125000

Rank 3: local sum = 218750125000

Rank 2: local sum = 156250125000

Rank 0: local sum = 31250125000

[Reduce] Total sum = 500000500000

[Reduce] Correct? = YES

[Reduce] Time = 0.006183 seconds

mpirun -np 8 ./sum_reduce

Root filled array with values 1 to 1000000

Rank 2: local sum = 39062562500

Rank 1: local sum = 23437562500

Rank 4: local sum = 70312562500

Rank 3: local sum = 54687562500

Rank 0: local sum = 7812562500

Rank 6: local sum = 101562562500

Rank 7: local sum = 117187562500

Rank 5: local sum = 85937562500

[Reduce] Total sum = 500000500000

[Reduce] Correct? = YES

[Reduce] Time = 0.006155 seconds

Exercise 05:

mpirun -np 2 ./sum_allreduce

Root filled array with values 1 to 1000000

Rank 0: local sum = 125000250000

Rank 1: local sum = 375000250000

Rank 0: local_sum = 125000250000, total_sum = 500000500000, contribution = 25.00%

Rank 1: local_sum = 375000250000, total_sum = 500000500000, contribution = 75.00%

[Allreduce] Correct? = YES
[Allreduce] Time = 0.000495 seconds

mpirun -np 4 ./sum_allreduce

Root filled array with values 1 to 1000000
Rank 2: local sum = 156250125000
Rank 1: local sum = 93750125000
Rank 0: local sum = 31250125000
Rank 0: local_sum = 31250125000, total_sum = 500000500000, contribution = 6.25%
Rank 3: local sum = 218750125000
Rank 3: local_sum = 218750125000, total_sum = 500000500000, contribution = 43.75%
Rank 2: local_sum = 156250125000, total_sum = 500000500000, contribution = 31.25%
Rank 1: local_sum = 93750125000, total_sum = 500000500000, contribution = 18.75%

[Allreduce] Correct? = YES
[Allreduce] Time = 0.005523 seconds

mpirun -np 8 ./sum_allreduce

Root filled array with values 1 to 1000000
Rank 1: local sum = 23437562500
Rank 6: local sum = 101562562500
Rank 0: local sum = 7812562500
Rank 7: local sum = 117187562500
Rank 3: local sum = 54687562500
Rank 5: local sum = 85937562500
Rank 2: local sum = 39062562500
Rank 4: local sum = 70312562500
Rank 6: local_sum = 101562562500, total_sum = 500000500000, contribution = 20.31%
Rank 0: local_sum = 7812562500, total_sum = 500000500000, contribution = 1.56%

[Allreduce] Correct? = YES
Rank 4: local_sum = 70312562500, total_sum = 500000500000, contribution = 14.06%
Rank 2: local_sum = 39062562500, total_sum = 500000500000, contribution = 7.81%
Rank 7: local_sum = 117187562500, total_sum = 500000500000, contribution = 23.44%
Rank 5: local_sum = 85937562500, total_sum = 500000500000, contribution = 17.19%
Rank 3: local_sum = 54687562500, total_sum = 500000500000, contribution = 10.94%
Rank 1: local_sum = 23437562500, total_sum = 500000500000, contribution = 4.69%

[Allreduce] Time = 0.005636 seconds

Exercise 06:

mpirun -np 2 ./sum_scan

Root filled array with values 1 to 1000000

Rank 0: local sum = 125000250000

Rank 0: local_sum = 125000250000, prefix_sum = 125000250000, sum_before_me = 0

Rank 1: local sum = 375000250000

Rank 1: local_sum = 375000250000, prefix_sum = 500000500000, sum_before_me = 125000250000

[Scan] Final prefix_sum on last rank = 500000500000

[Scan] Correct? = YES

[Scan] Time = 0.003231 seconds

mpirun -np 4 ./sum_scan

Root filled array with values 1 to 1000000

Rank 3: local sum = 218750125000

Rank 2: local sum = 156250125000

Rank 0: local sum = 31250125000

Rank 0: local_sum = 31250125000, prefix_sum = 31250125000, sum_before_me = 0

Rank 1: local sum = 93750125000

Rank 1: local_sum = 93750125000, prefix_sum = 125000250000, sum_before_me = 31250125000

Rank 2: local_sum = 156250125000, prefix_sum = 281250375000, sum_before_me = 125000250000

Rank 3: local_sum = 218750125000, prefix_sum = 500000500000, sum_before_me = 281250375000

[Scan] Final prefix_sum on last rank = 500000500000

[Scan] Correct? = YES

[Scan] Time = 0.004346 seconds

mpirun -np 8 ./sum_scan

Root filled array with values 1 to 1000000

Rank 5: local sum = 85937562500

Rank 4: local sum = 70312562500
 Rank 2: local sum = 39062562500
 Rank 1: local sum = 23437562500
 Rank 6: local sum = 101562562500
 Rank 7: local sum = 117187562500
 Rank 0: local sum = 7812562500
 Rank 0: local_sum = 7812562500, prefix_sum = 7812562500, sum_before_me = 0
 Rank 1: local_sum = 23437562500, prefix_sum = 31250125000, sum_before_me = 7812562500
 Rank 3: local sum = 54687562500
 Rank 3: local_sum = 54687562500, prefix_sum = 125000250000, sum_before_me = 70312687500
 Rank 2: local_sum = 39062562500, prefix_sum = 70312687500, sum_before_me = 31250125000
 Rank 4: local_sum = 70312562500, prefix_sum = 195312812500, sum_before_me = 125000250000
 Rank 5: local_sum = 85937562500, prefix_sum = 281250375000, sum_before_me = 195312812500
 Rank 6: local_sum = 101562562500, prefix_sum = 382812937500, sum_before_me = 281250375000
 Rank 7: local_sum = 117187562500, prefix_sum = 500000500000, sum_before_me = 382812937500

[Scan] Final prefix_sum on last rank = 500000500000

[Scan] Correct? = YES

[Scan] Time = 0.004206 seconds

Exercise	Approach	2 procs	4 procs	8 procs	2→8 change
01	Bcast	0.0016	0.0046	0.0075	×4.7
02	Scatter	0.0012	0.0058	0.0039	×3.3
03	Gather	0.001814	0.005815	0.005598	×3.1
04	Reduce	0.002309	0.006183	0.006155	×2.7
05	Allreduce	0.000495	0.005523	0.005636	×11.4
06	Scan	0.003231	0.004346	0.004206	×1.3

Fastest approach: Allreduce wins at 2, Bcast at 4, Scatter at 8. The spread at any given count is 1-3 ms on single runs, so that ordering is almost certainly noise rather than a genuine ranking.

3. MPI_Scan: when global indexing or dynamic load balancing needed.

Concrete Example: Imagine each process parses a different chunk of a log file and extracts a variable number of error strings. They all want to write their extracted errors into a single, contiguous global array. By using MPI_Scan on the count of errors each process found, every process learns exactly what index offset it should start writing at (sum_before_me), preventing them from overwriting each other's data. MPI_Allreduce would only tell them the total number of errors, but wouldn't help them coordinate where to write.